ZDEEP iOS: привязка автомобиля, сессия и WebView

Дата: 2026-10-03.

Статус: проект спецификации по согласованным в разговоре архитектуре, контракту и переходам. Требует проверки письменного документа перед планом реализации.

Уточнение 2026-10-03: спецификация сверена с реализованной инфраструктурой локальной ветки `main`, включая коммиты до `252e0c7`. Сравнение выполнено по исходникам и отчёту инфраструктуры, без нового прогона приложения. Удалённая ветка не проверялась: в текущем checkout не настроен Git remote.

1. Цель и предпосылки

Новый пользователь сканирует QR на экране автомобиля, регистрирует ключ установки приложения и получает пользовательский JWT. Приложение показывает веб-интерфейс машины и поддерживает его сессию. При следующих запусках вход выполняется через сохранённую привязку без нового QR.

К началу реализации считаем готовыми инфраструктуру сборки, тестов и диагностики и нативный welcome screen с кнопкой «Сканировать QR». Реализация веб-части и bridge по отдельному заданию — обязательная зависимость до начала реализации интеграции iOS.

Инфраструктура уже присутствует в `main`; её подтверждения описаны в отчёте проверки. Welcome с QR остаётся отдельной предпосылкой: текущий `ContentView.swift` содержит инфраструктурный экран с «Открыть WebView», а не готовый продуктовый welcome. Эта спецификация не включает повторную реализацию инфраструктуры и не объявляет welcome уже готовым в `main`.

Нативная загрузка в первой версии содержит только текст «Загрузка» и кнопку «Отмена». Её оформление заменяется позднее. Нативные ошибки используют существующий экран ошибки из Stitch с действиями, описанными ниже. Макеты и стили переносятся по проектным навыкам; новые визуальные значения не подбираются самостоятельно.

Рабочий проект Stitch: `14173823773588150 87`. Прочитаны актуальные HTML welcome `bc93291bf18e4d78b737a9a035b30434` и ошибки `411b28d9024448b7a36d813cbf00b56c`. В последнем есть заголовок, объяснение, диагностический код и инструкция перезапуска; инструкцию заменяем согласованными действиями. Финальный экран при переносе снова получаем из Stitch, особенно если он был изменён.

В эту версию не входят серверный отзыв сертификатов, управление несколькими машинами в одном приложении и гарантированное периодическое выполнение в фоне. Сервер продолжает принимать прежние сертификаты после локального сброса.

2. Архитектура

Компонент	Ответственность	Зависимости
Координатор приложения	Видимый экран, камера, переходы, отмена и сброс	Хранилище привязки, менеджер сессии, контейнер WebView
Хранилище привязки	Ключи и метаданные, состояние регистрации, очистка незавершённых попыток	Keychain и локальные метаданные установки

Клиент авторизации	`pair` и `token`, подпись, валидация ответа, таймаут	Ключ установки, транспорт, готовая конфигурация инфраструктуры
Менеджер сессии	JWT в памяти, срок действия, обновление и повторы	Клиент авторизации, часы и планировщик, жизненный цикл
Контейнер WebView и bridge	Ранняя загрузка страницы, веб-кеш, сессия для JS, готовность документа	Готовая конфигурация инфраструктуры, менеджер сессии, координатор

Состояния привязки, сессии и WebView независимы. Координатор не определяет завершение регистрации по наличию приватного ключа. Получение JWT не означает готовность веб-интерфейса, а истечение JWT не закрывает уже показанный WebView.

Все изменения состояния сериализуются. Асинхронный результат применяется только к актуальным привязке, попытке и документу WebView. Проверки отменённой попытки и старого документа остаются внутренней защитой координатора; отдельной ревизии JWT в модели сессии и bridge нет. В любой момент разрешён один запрос авторизации: плановое обновление, запрос bridge и пользовательский повтор присоединяются к актуальной операции либо используют уже полученную сессию.

2.1. Подключение к реализованной инфраструктуре

Расширяем существующие компоненты, не создаём второй параллельный координатор или контейнер WebKit:

Готовая часть	Изменение для бизнесового сценария
`App/AppCoordinator.swift`	Добавить состояния экранов и зависимости авторизации. Разделить подготовку `activeWebSession` и показ WebView: текущий `openWebView()` одновременно создаёт, загружает и открывает его.
`Web/WebSession.swift`, `Web/WebViewHost.swift`	Сохранить экземпляр WebKit, наблюдение навигации, загрузку, перезагрузку и диагностическое выполнение JS. Заменить инжекцию пустого `ZdeepNative` полноценным bridge.
`Environment/AppEnvironment.swift`, `LaunchConfiguration.swift`	Получать адреса и уведомления об изменении конфигурации из готовой инфраструктуры.
`Debug/DiagnosticMailbox.swift`, `DebugPanel.swift`	Привязать диагностику к существованию подготовленного WebView, независимо от того, показан ли он пользователю. Сохранить инвалидацию при закрытии/замене экземпляра.
`Debug/DiagnosticEventStore.swift`	Использовать существующий фильтрованный журнал; добавить безопасные события авторизации, обновления токена и готовности без секретных значений.

`ZDEEPApp.swift`	Подключить активность приложения к менеджеру сессии и бюджету готовности; сейчас бизнесовая обработка `scenePhase` отсутствует.

Текущий `isWebViewOpen` не может одновременно означать наличие подготовленного экземпляра и разрешение его показать. UI представляет один и тот же `WKWebView` за непрозрачным нативным экраном, до готовности исключает его из взаимодействия и accessibility. `activeWebSession` остаётся доступной диагностике. Последующее раскрытие не создаёт новый экземпляр и не загружает URL повторно.

`WebSession.id` идентифицирует экземпляр WebKit и USB mailbox. Координатор также отсеивает результаты отменённых попыток и старых документов. `NavigationCallbackGate` уже защищает навигационные callbacks; бизнесовые bridge-вызовы должны проверять свою актуальность перед применением результата. Эти проверки не требуют поля версии сессии в JWT или веб-контракте.

Нативный бизнесовый bridge должен работать в Debug и Release. `DiagnosticExecutor`, произвольное диагностическое исполнение JS, debug UI и USB mailbox остаются только в Debug. События сессии, ответы bridge и `reportReady` реализуются независимо от `executeJavaScript()` инфраструктуры, который сейчас исключён из Release.

3. Постоянная привязка и JWT

Постоянная запись содержит:

- идентификатор локальной привязки и ссылку на приватный ключ;
- публичный ключ в серверном формате `publicCertificate`;
- `carId`, подтверждённый ответом сервера;
- состояние `pending`, `registered` или `resetting`;
- метаданные принадлежности записи, используемые существующей инфраструктурой очистки.

Приватный ключ хранится в Keychain, не передаётся вебу и серверу. Для первой версии предлагается P-256 через CryptoKit, сериализованный приватный ключ в Keychain через Security framework, без обязательного биометрического диалога. Доступность — `AfterFirstUnlockThisDeviceOnly`, без синхронизации iCloud: это позволяет завершать начатую операцию после блокировки в пределах времени, предоставленного iOS. Метаданные регистрации также должны быть доступны для такого завершения. Конкретные идентификаторы Keychain определяются планом реализации.

В API словом «сертификат» назван публичный EC-ключ P-256 в DER/SPKI и Base64. Выпуск X.509 не требуется. Публичный ключ сохраняется в каноническом формате, принимаемом бэкендом.

Перед запросом `pair` создаём новую `pending`-привязку. Только получение и валидация успешного ответа с JWT позволяет сохранить `registered`. Сначала надёжно сохраняем завершённую запись, затем публикуем сессию для веба. Если сохранение не удалось, регистрация считается неудачной. Ответ должен содержать непустой JWT, `userId`, согласованный `carId` и корректный срок действия; значения, необходимые вебу, проверяются до публикации.

На старте удаляем остатки `pending`-попытки и относящиеся к ней ключи; для `resetting` завершаем очистку. Полная `registered`-привязка используется для `token`. Отсутствующий ключ не позволяет восстановить привязку и возвращает на welcome. Временная недоступность Keychain не приравнивается к отсутствию ключа: показываем ошибку и сохраняем запись для повтора.

JWT хранится только в памяти нативного приложения и веб-сессии. Новый запуск процесса всегда получает JWT через `token`; диск и веб-хранилища для восстановления JWT не используются. Срок действия берётся из ответа сервера и `exp`, а не из постоянного клиентского значения.

4. QR и протокол авторизации

QR агента содержит URL вида `<BASE_URL>auth/get-otp?token=<JWT машины>`. Извлекаем один непустой параметр `token`, проверяем поддерживаемую структуру ссылки и токена. URL из QR не становится адресом API или WebView: оба адреса берутся из конфигурации приложения. Сервер проверяет подлинность и срок JWT машины. Данные, декодированные из QR на телефоне, не считаются подтверждённой сервером привязкой.

Сканер принимает первое распознанное значение, прекращает сканирование и закрывает камеру. Неподдерживаемый QR приводит к ошибке первичной регистрации с «Начать сначала». Прочие QR не открываются как ссылки.

Локальный референс API:

Метод	Тело JSON
`POST /auth/mobile/pair`	`publicCertificate`, `carToken`
`POST /auth/mobile/token`	`publicCertificate`, `carId`

Передаются `Content-Type: application/json`, `X-Timestamp` и `X-Signature`. Подписываются точные UTF-8 байты строки `<timestamp>|<rawBody>`: ECDSA P-256 с SHA-256, подпись DER, заголовок в Base64. Timestamp — десятичные миллисекунды. JSON сериализуется один раз; подписанные байты совпадают с отправленными. Каждый новый запрос получает новый timestamp и подпись.

Успешный ответ референса содержит `accessToken`, `tokenType`, `expiresIn`, `serverTime`, `userId`, `carId`. Текущий срок JWT — 900 секунд. В bridge преобразуем срок в UTC ISO `expiresAt`. Учитываем серверное время и фактическую доставку, консервативно оценивая оставшийся срок; во время работы процесса таймеры используют монотонные часы. Время подписи также учитывает известную поправку часов сервера. Первичная регистрация не получает скрытого повтора даже при `TIMESTAMP_OUT_OF_RANGE`; доступную поправку можно использовать в следующей пользовательской попытке.

Адреса API и WebView предоставляет готовая инфраструктура. Реализация авторизации использует её конфигурацию и не дублирует правила окружений, префиксов маршрутов и выбора адресов.

5. Первый вход и ранняя загрузка WebView

При запуске создаём WebView и начинаем загрузку страницы параллельно с нативным сценарием. До успешного входа он расположен за welcome, камерой или нативной загрузкой. Bridge внедряется на старте документа до выполнения скриптов веб-приложения. Пока сессии нет, первоначальный `getSession()` ожидает её; веб не отправляет запросы данных машины без действующего JWT.

После получения JWT сессия становится доступной bridge. С этого момента отсчитываем до 60 секунд активного времени приложения на готовность текущего документа. Отсчёт не зависит от того, успел ли зависший JS вызвать `getSession()`: готовый JWT уже подготовлен для передачи. Паузы в фоне в этот бюджет не входят.

Веб сообщает готовность только после отображения данных машины или обработанного экрана ошибки. Сигнал относится к текущему документу. Успешная навигация WebKit сама по себе не заменяет этот сигнал. Координатор принимает его только при актуальной привязке, действующей сессии и продолжающемся ожидании готовности, после чего показывает WebView.

Если веб сообщил готовность до получения JWT, от отменённой попытки или прежнего документа, сигнал не открывает экран. Если ожидание уже завершилось ошибкой, поздний сигнал сам не убирает экран ошибки: пользователь нажимает «Повторить».

Ошибка ранней загрузки веба до начала регистрации не заменяет welcome или камеру. Она учитывается, когда потребуется показать WebView: выполняем загрузку в рамках текущего входа либо показываем ошибку её актуальной попытки.

6. Согласованные переходы

Ситуация	Действие	Видимый экран
Запуск без завершённой привязки	Загрузка веба за нативным экраном	Welcome
«Сканировать QR», доступ не запрашивали	Системный запрос разрешения	Диалог iOS
Доступ получен или уже разрешён	Открыть сканер	Камера
Отказ или уже запрещённый доступ	Объяснение	«Открыть настройки», «Назад»
Закрытие сканера или «Назад»	Завершить сканирование	Welcome
QR считан	Создать ключи, отправить `pair`	«Загрузка», «Отмена»
Первый JWT получен	Сохранить регистрацию, передать сессию вебу	«Загрузка», «Отмена»
Актуальный сигнал готовности	Убрать нативную загрузку	WebView
Первичная регистрация не удалась	Удалить незавершённую привязку	Причина ошибки, «Начать сначала»
«Начать сначала»	Возврат к началу	Welcome
Запуск с завершённой привязкой	Параллельно загрузить веб и запросить `token`	«Загрузка», «Отмена»
`token` при запуске не получен	Сохранить привязку	Ошибка, «Повторить», «Привязать машину заново»
Веб не загрузился или готовность не пришла за 60 активных секунд	Сохранить завершённую привязку	Ошибка с теми же двумя кнопками

«Повторить»	Повторить нужный этап; получить JWT, если действующего нет	«Загрузка», «Отмена»
«Отмена» на любой нативной загрузке	Остановить операции, удалить привязку и JWT	Welcome
«Привязать машину заново»	Предупредить о локальном сбросе и новом QR	Подтверждение
Сброс подтверждён	Выполнить очистку	Welcome
Сброс отклонён	Сохранить состояние	Прежняя ошибка
До истечения JWT осталось ≤2 минут	Начать обновление	Показанный WebView остаётся видимым
Обновление не удалось	Запланировать повтор	WebView
JWT истёк	Уведомить веб, продолжать попытки	WebView с веб-ошибкой
Новый JWT получен	Опубликовать сессию, уведомить веб	WebView восстанавливает данные
Уход в фон	Приостановить повторы и бюджет готовности	Приложение в фоне
Возвращение	Проверить срок, обновить при необходимости	Сохранить текущий экран
Процесс завершён iOS	На новом старте использовать `registered`, очистить `pending`	Обычный сценарий запуска

Для ограничений камеры, установленных системой, объяснение не обещает, что разрешение обязательно можно изменить в настройках. После возвращения из настроек состояние разрешения читается повторно.

7. Ошибки и повторы

До первого JWT любой неуспех `pair`, некорректный QR, ошибка ключей или некорректный ответ очищает незавершённую привязку и открывает ошибку с «Начать сначала». Автоматических повторов `pair` нет. Сервер мог успеть создать свою запись: локальная очистка её не отзывает, последующая попытка создаёт новые ключи.

После завершённой регистрации ошибка первого `token` при запуске не очищает ключ и не запускает бесконечные скрытые повторы за нативной ошибкой. «Повторить» выполняет одну новую попытку входа. Ошибка навигации или готовности после регистрации также сохраняет ключ; повторяется загрузка страницы и ожидание готовности, при необходимости обновляется JWT.

После первого показа WebView менеджер сессии повторяет обновление до успеха. В текущей версии отрицательный ответ авторизации не вызывает автоматического удаления ключа и отдельного процесса отзыва. В локальных исходниках уже есть `UNKNOWN_USER` и `CAR_ACCESS_DENIED`, но

продуктовый процесс отзыва пользователем не включён в эту версию: такие ответы обрабатываются как неуспех обмена, без очистки сертификата.

Параметры:

Параметр	Значение
Начало планового обновления	За 120 секунд до истечения
Максимальное время одного запроса	30 секунд
Паузы после последовательных неудач	5, 10, 20, 30 секунд, далее 30 секунд
После истечения JWT	Те же попытки продолжаются
После успеха	Сброс счётчика неудач, новый план по новому сроку
Бюджет первого показа веба	60 секунд активного времени после готовности сессии к передаче

Пауза начинается после завершения неудачного запроса. Срок JWT проверяется независимо от выполняющегося запроса: если токен истёк во время 30-секундного ожидания, веб получает уведомление без ожидания ответа. Запрос, начатый до истечения и успешно завершившийся после него, публикует новую сессию обычным образом. Сетевой запрос после паузы получает новую подпись. Фиксированная схема пауз не дополняется случайным разбросом в этой версии.

Временный отказ обновления до истечения JWT не скрывает веб и не сообщает ему, что текущая сессия уже потеряна. После истечения веб показывает восстанавливаемую ошибку. При успешном обновлении bridge отправляет `sessionChanged` с полной новой сессией: `accessToken`, `carId`, `expiresAt`. Веб принимает токен из события без дополнительного чтения и перезагрузки страницы. Новый JWT восстанавливает чтение данных, но не повторяет автоматически операции изменения данных с неопределённым результатом.

Пользовательские сообщения различают отсутствие связи, таймаут, ошибку сервера, неподдерживаемый/просроченный QR, ошибку локального ключа и загрузки веба. Для ответа авторизации экран показывает русское объяснение и диагностическую строку с HTTP-статусом и серверным `code`, например: «Доступ к автомобилю запрещён» и `HTTP 403 · CAR_ACCESS_DENIED`. Бэкенд сейчас возвращает JSON с `code`, а не готовое пользовательское сообщение. Неизвестный код также отображается; при отсутствии корректного JSON показывается HTTP-статус и общее объяснение. Сырой HTML и произвольное тело ответа в интерфейс не подставляются.

Серверный код	Объяснение пользователю
`INVALID_CAR_TOKEN`	QR-код недействителен или истёк; требуется новый QR
`UNKNOWN_USER`	Сервер не нашёл пользователя для этой привязки
`CAR_ACCESS_DENIED`	Доступ к автомобилю запрещён
`INVALID_SIGNATURE`	Сервер отклонил подпись запроса
`TIMESTAMP_OUT_OF_RANGE`	Время запроса не соответствует времени сервера

`RATE_LIMITED`	Слишком много запросов авторизации
`AUTH_UNAVAILABLE`, `INTERNAL_ERROR`	Сервис авторизации временно недоступен или завершился ошибкой
`INVALID_REQUEST`, `METHOD_NOT_ALLOWED`, `UNSUPPORTED_MEDIA_TYPE`	Сервер отклонил формат запроса авторизации

Показ кода не меняет согласованные действия: первичная регистрация — «Начать сначала», вход с сохранённым ключом — «Повторить» и «Привязать машину заново». При работающем WebView отказ обновления до истечения не прерывает интерфейс; после истечения или неудачного восстановления через bridge веб получает описание последней ошибки для своего экрана. Последующий JWT восстанавливает данные. Привязка автоматически не удаляется по коду ошибки.

Диагностика содержит этап, HTTP-статус, серверный код и локальный идентификатор; QR JWT, пользовательский JWT, подписи и приватные ключи не выводятся в логи и текст ошибок.

8. Отмена и сброс

Кнопка «Отмена» существует на всём нативном экране загрузки, в том числе после сохранения регистрации и до первого показа веба. Она выполняет локальный сброс без отдельного подтверждения. Сброс с экрана ошибки требует подтверждения: «Текущая привязка на телефоне будет сброшена. Чтобы подключить автомобиль снова, потребуется отсканировать QR-код».

Порядок очистки:

1. Инвалидировать поколение привязки, попытки, сообщения и ожидающие ответы.
2. Остановить авторизационный запрос, повторы и ожидание готовности.
3. Надёжно сохранить состояние `resetting`, чтобы перезапуск не восстановил её.
4. Удалить принадлежащие этой привязке ключи, метаданные и JWT.
5. Завершить текущий документ веба с его JS-сессией и данными машины.
6. Вернуться на welcome и подготовить новый WebView для ранней загрузки.

Сброс не отправляет серверный отзыв и не открывает камеру автоматически. Поздний успех `pair`, `token` или `reportReady()` не может восстановить удалённую привязку. Ошибка физического удаления ключа не маскируется успешным сбросом: запись остаётся исключённой из входа, очистка завершается до новой регистрации.

Полный сброс через debug

Debug-действие «Сбросить данные приложения» очищает все данные, принадлежащие ZDEEP на устройстве, включая Keychain. Это отдельная операция от отмены и перепривязки, сохраняющих кеш статических ассетов.

1. Остановить запросы, таймеры, ожидание готовности и инвалидировать текущие результаты.
2. Завершить WebView и очистить JWT, сессию и данные машины в памяти.
3. Удалить все Keychain-записи приложения: зарегистрированные и незавершённые ключи, остатки неудачной очистки и другие принадлежащие приложению секреты. Очистка одной текущей ссылки на ключ недостаточна. Keychain других приложений не затрагивается.
4. Очистить весь домен UserDefaults приложения, локальные записи привязки, файлы и кеши приложения, а также все типы данных `WKWebsiteDataStore.default()`.
5. Перейти в состояние нового запуска без привязки. Если удаление не удалось, показать ошибку сброса; не объявлять полный сброс успешным и не использовать оставшуюся

привязку.

Готовый `resetClientData()` / `clearOwnedClientState()` уже удаляет данные WebKit и UserDefaults. Его дополняем очисткой Keychain, файлов и оперативного состояния по этому списку. Закрытие WebView средствами диагностики само по себе не удаляет привязку.

9. Фон и возвращение

Периодические запросы JWT запускаются только при активном приложении. На уходе в фон отменяем запланированные будущие повторы, приостанавливаем 60-секундный бюджет. Уже начатую операцию можно завершить в ограниченное время через системный механизм продления фоновой работы. Блокировка или сворачивание сами по себе не означают пользовательскую отмену.

Если iOS приостановила выполнение, на возвращении сверяем фактическое состояние, срок запроса и JWT. Просроченный авторизационный запрос завершается как таймаут; он не запускается заново как часть первичной регистрации. Если сохранён успешный результат, используем его. После завершения процесса применяется правило `pending` / `registered` из раздела 3.

Для показанного WebView при возврате с истёкшим JWT или остатком ≤120 секунд делаем немедленную попытку, без ожидания ранее назначенной паузы. При неудаче продолжается последовательность пауз текущей серии. Если актуальный запрос уже выполняется, второй не создаётся. Для JWT с большим остатком восстанавливается плановое обновление. Нативное представление WebView при этом не закрывается.

Background App Refresh для регулярного обновления JWT не требуется. Не обещаем работу таймеров в приостановленном процессе.

10. WebView, кеш и границы документов

Используем постоянное хранилище `WKWebsiteDataStore.default()`. Приложение не очищает статический кеш при обычном запуске, обновлении JWT, отмене или перепривязке. В течение одной сессии обновление JWT не пересоздаёт WebView и не перезагружает страницу.

Готовый `WebSession` уже создаёт WebKit с конфигурацией, использующей постоянное хранилище по умолчанию. Собственный кеш статических файлов ради этой задачи не создаётся; новая логика сохраняет это хранилище и разделяет способы очистки.

Сохранение кеша не равно сохранению экземпляра WebView после завершения процесса и не гарантирует, что каждый ресурс останется на диске. Веб-сервер задаёт корректное кеширование версионированных JS/CSS/изображений и проверку актуальности HTML. Пользовательские данные и JWT не попадают в постоянный кеш веб-приложения. Полный debug-сброс обязательно очищает веб-данные вместе с остальными данными приложения.

Каждый новый документ получает транспортную идентичность. Bridge доступен только главному frame доверенного origin из готовой конфигурации. Смена документа, экземпляра WebView или привязки делает предыдущие JS-ответы недействительными. Внешняя навигация не получает нативную сессию. Отменённая навигация не отображается как пользовательская ошибка.

Гибель веб-процесса до первого показа обрабатывается как ошибка загрузки с сохранением регистрации. После первого показа сохраняем нативную привязку и контейнер веба, восстанавливаем документ и передаём текущую сессию; при отсутствии токена продолжаем обмен. Кеш может помочь восстановлению, но прежнее JS-состояние гарантировать нельзя.

Настройки окружений, переключение адресов и правила сохранения либо сброса данных остаются ответственностью готовой инфраструктуры. Авторизация получает её конфигурацию; при замене WebView или сигнале сброса прекращает прежние операции и инвалидирует их ответы. В этой функциональности нет отдельной логики выбора окружения.

Готовность USB mailbox означает готовность диагностического транспорта и не заменяет `reportReady`. Симуляторный `diagnose` не должен обходить авторизацию или помечать бизнесовый экран готовым. Debug-перезагрузка инвалидирует документ и ожидающие bridge-ответы, сохраняя привязку.

В текущем `WebSession` разрешены все навигации, а `webViewWebContentProcessDidTerminate` не обрабатывается. Ограничение origin и обработка гибели веб-процесса из этой спецификации входят в будущую реализацию, не считаются уже обеспеченными инфраструктурой.

11. Приёмка будущей реализации

Это требования к проверкам будущей реализации, а не отчёт о выполненных тестах.

- Первый вход: разрешение камеры, закрытие, отказ и настройки, корректный и неподдерживаемый QR, успешный `pair`.
- Ошибки до первого JWT: нет связи, таймаут, HTTP-ошибка, некорректный ответ; один запрос, очистка ключа, «Начать сначала».
- Отмена до JWT и после JWT во время подготовки веба; запоздавшие сетевые ответы и готовность не меняют welcome.
- Повторный запуск получает JWT с той же привязкой; ошибка сохраняет ключ; повтор и подтверждённый сброс работают.
- Ранняя загрузка веба происходит параллельно авторизации; без JWT нет запросов данных машины.
- Веб показывается только по актуальному `reportReady()`; данные и обработанная ошибка оба удовлетворяют готовности.
- 60 секунд считаются только в активном приложении; зависший JS, потерянный сигнал и ошибки навигации дают нативную ошибку.
- Контроль времени подтверждает начало за 120 секунд, паузы 5/10/20/30, отсутствие параллельных запросов, истечение во время запроса и повторы после истечения.
- Показанный веб не скрывается при истечении JWT; обновление восстанавливает данные без перезагрузки страницы и без повторного выполнения мутаций.
- Пропущенные события, возврат из фона, старые документы и запоздавшие ответы не нарушают состояние.
- Завершение процесса с `pending` и с `registered` даёт разные согласованные сценарии запуска.
- Перезапуск приложения использует постоянный кеш при разрешающих заголовках сервера; новая привязка не отображает данные прежней машины.
- Реальный локальный бэкенд принимает SPKI и подпись точного тела, JWT веба работает; интеграции используют уже настроенную тестовую инфраструктуру по навыку `ios-testing`.

Часы, транспорт и планировщик имеют заменяемые зависимости для проверок таймаутов и гонок без ожидания 15 минут. Реальный iPhone нужен для разрешения камеры, блокировки, фонового завершения и поведения WebKit.

Готовые `WebSessionIntegrationTests` проверяют реальную навигацию, наличие объекта `ZdeepNative`, reinjection, исполнение JS и отсечение запоздавших ответов; они пока не проверяют авторизацию или готовность данных машины. Эти проверки сохраняются и расширяются бизнесовым

контрактом. Файловая HTML-фикстура остаётся средством проверки инжекции без предоставления реальных ключей или JWT недоверенному origin. Инфраструктурные UI-тесты «Открыть/Закрыть WebView» адаптируются к бизнесовым экранам и отдельным debug-действиям, не удерживают прежний интерфейс в Release.

Дополнительные проверки интеграции с инфраструктурой: подготовленный веб доступен диагностике до первого показа; раскрытие не создаёт второй экземпляр; очистка привязки сохраняет статический кеш, а полный debug-сброс удаляет также Keychain; mailbox readiness не открывает бизнесовый экран; замена конфигурации и reload инвалидируют ожидающие ответы; бизнесовый bridge присутствует в Release без debug API. Фон и блокировка проверяются отдельно от USB-команд, которым нужен foreground.

12. Готовые библиотеки и системные API

Варианты изучены через Context7 и первичную документацию проектов. Это рекомендация к выбору зависимостей, а не утверждение об установленном пакете или проверенной совместимости подписи.

Задача	Вариант	Решение для первой версии
QR-сканер SwiftUI	CodeScanner поверх AVFoundation: одно сканирование, ограничение `.qr`, симуляция результата	Предлагается как единственная внешняя зависимость: сокращает код камеры. Разрешения, закрытие и переходы остаются у координатора. Перед добавлением сверить доступные параметры и платформы выбранного релиза.
Нативное сканирование	AVCaptureMetadataOutput; VisionKit DataScanner	AVFoundation — вариант без пакетов. DataScanner требует проверок `isSupported`/`isAvailable`, поэтому для универсального QR-сценария не является первым выбором.
P-256 и подпись	CryptoKit P256.Signing	Использовать системный API. Публичный ключ — `derRepresentation`, подпись — DER; сериализованный приватный ключ хранить в Keychain. Совместимость SPKI и подписи с бэкендом подтвердить интеграцией.
Keychain	Security; KeychainAccess	Начать с небольшого адаптера Security. KeychainAccess — готовая альтернатива для хранения сериализованного ключа как Data; он не генерирует P-256. Его `removeAll()` ограничен областью настроенного хранилища и сам по себе не обеспечивает полный сброс всех записей приложения.
Два запроса авторизации	URLSession; Alamofire	Использовать URLSession: отдельная HTTP-библиотека для двух методов пока не оправдана. Отмена, 30-секундный таймаут и согласованные повторы принадлежат

		менеджеру сессии; автоматических повторов `pair` нет.
JWT	Ответ сервера и минимальное чтение нужных полей	Клиент не выпускает JWT. Отдельная библиотека подписи JWT не требуется; декодирование не заменяет серверную проверку.

Предложение по зависимостям: CodeScanner для камеры, CryptoKit + Security + URLSession для авторизации, существующий контейнер WebKit для веба. Окончательный выбор QR-пакета фиксируется перед планом реализации; в рамках этой правки зависимости не добавляются.

13. Источники и следующий этап

Прочитанные референсы:

- `/Users/oleg/projects/zdeep-webview2/backend/README.md`, `backend/http-server.mjs` — локальные маршруты и серверный адаптер.
- `/Users/oleg/projects/zdeep/packages/cloud-functions/src/services/MobileAuthService/` и `src/entrypoints/auth/mobile/mobile.ts` — точный формат ключа, подписи, ответа и ошибки.
- `/Users/oleg/projects/zdeep-agent/app/src/main/java/com/zdeep/agent/utils/QRCodeService.kt`, `JwtTokenGenerator.kt` — URL QR, ротация и JWT машины.
- `/Users/oleg/projects/zdeep-webview2/src/shared/session/` и `src/app/session/SessionProvider.tsx` — текущая сессия и восстановление.
- Спецификация инфраструктуры.
- Отчёт проверки реализованной инфраструктуры, протокол диагностики iPhone.
- `ZDEEP/ZDEEP/App/AppCoordinator.swift`, `Web/WebSession.swift`, `Web/WebViewHost.swift`, `Environment/LaunchConfiguration.swift`, `Environment/AppEnvironment.swift`, `Debug/DiagnosticMailbox.swift` — точки подключения готовой инфраструктуры.

Документация Apple искалась через Context7; ключевые первичные источники:

- Постоянное хранилище WebKit.
- Инжекция на старте документа.
- Ограниченное завершение работы в фоне.
- Планирование фоновых задач.
- Доступность ключей после первой разблокировки.

После проверки и одобрения этой письменной спецификации составляется план реализации. Веб-задание передаётся отдельному исполнителю; интеграция iOS начинается после готовности его контракта. Этот документ не разрешает автоматически начинать реализацию или изменять соседние репозитории.